



Aalto University

Teaching masked diffusion language models to generate chess puzzles with reinforcement learning

Aatu Selkee

July 3, 2026

Advisor Adjunct Prof. Eric Malmi

Supervisor Prof. Nuutti Hyvönen

Table of Contents

Introduction

Background

Methodology

Results

Conclusion

Example puzzles

Introduction

Introduction

- ▶ In this thesis, we train a model to generate chess puzzles with certain properties.
- ▶ Chess puzzle generation requires creativity and reasoning.
- ▶ Contributions:
 - ▶ Demonstrate the effectiveness of masked diffusion models.
 - ▶ Demonstrate that chess puzzle generation can be conditioned on specific themes.
 - ▶ Demonstrate that masked diffusion models can be improved via reinforcement learning.
 - ▶ Demonstrate that auxiliary task of best move prediction improves puzzle generation.
- ▶ We follow the work of Feng et al..

Introduction

- ▶ In this thesis, we train a model to generate chess puzzles with certain properties.
- ▶ Chess puzzle generation requires creativity and reasoning.
- ▶ Contributions:
 - ▶ Demonstrate the effectiveness of masked diffusion models.
 - ▶ Demonstrate that chess puzzle generation can be conditioned on specific themes.
 - ▶ Demonstrate that masked diffusion models can be improved via reinforcement learning.
 - ▶ Demonstrate that auxiliary task of best move prediction improves puzzle generation.
- ▶ We follow the work of Feng et al..

Introduction

- ▶ In this thesis, we train a model to generate chess puzzles with certain properties.
- ▶ Chess puzzle generation requires creativity and reasoning.
- ▶ Contributions:
 - ▶ Demonstrate the effectiveness of masked diffusion models.
 - ▶ Demonstrate that chess puzzle generation can be conditioned on specific themes.
 - ▶ Demonstrate that masked diffusion models can be improved via reinforcement learning.
 - ▶ Demonstrate that auxiliary task of best move prediction improves puzzle generation.
- ▶ We follow the work of Feng et al..

Introduction

- ▶ In this thesis, we train a model to generate chess puzzles with certain properties.
- ▶ Chess puzzle generation requires creativity and reasoning.
- ▶ Contributions:
 - ▶ Demonstrate the effectiveness of masked diffusion models.
 - ▶ Demonstrate that chess puzzle generation can be conditioned on specific themes.
 - ▶ Demonstrate that masked diffusion models can be improved via reinforcement learning.
 - ▶ Demonstrate that auxiliary task of best move prediction improves puzzle generation.
- ▶ We follow the work of Feng et al..

Language modelling

Language modelling

- ▶ In language modelling, autoregressive language models are the most common paradigm currently.
- ▶ In recent years, diffusion models, have become very popular for continuous problems, such as image, video or audio generation.
- ▶ Discrete diffusion, in particular masked diffusion, has become a compelling alternative for large language models (LLMs).

Language modelling

- ▶ In language modelling, autoregressive language models are the most common paradigm currently.
- ▶ In recent years, diffusion models, have become very popular for continuous problems, such as image, video or audio generation.
- ▶ Discrete diffusion, in particular masked diffusion, has become a compelling alternative for large language models (LLMs).

Language modelling

- ▶ In language modelling, autoregressive language models are the most common paradigm currently.
- ▶ In recent years, diffusion models, have become very popular for continuous problems, such as image, video or audio generation.
- ▶ Discrete diffusion, in particular masked diffusion, has become a compelling alternative for large language models (LLMs).

Autoregressive LLM vs. Masked Diffusion

Autoregressive LLM

Step 1: Machines

Step 2: Machines take

Step 3: Machines take me

Step 4: Machines take me by

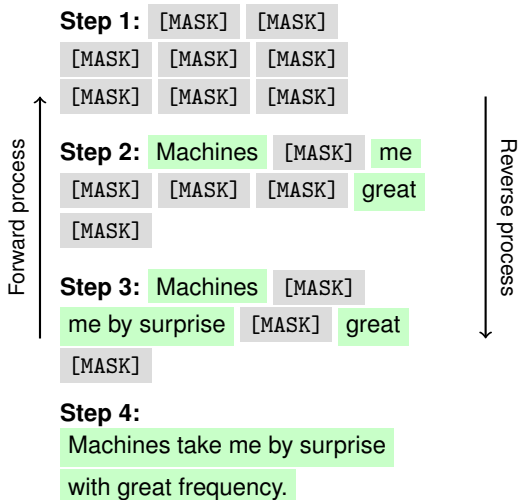
Step 5: Machines take me by surprise

...

Step 8: Machines take me by surprise with great frequency.

– Alan Turing

Masked Diffusion



Autoregressive LLM vs. Masked Diffusion

Table: Comparison of Autoregressive LLMs and Masked Diffusion Models.

	Autoregressive LLMs	Masked Diffusion Models
Generation Method	Left-to-right	Random order
Inference Speed	Linear in output length (Slow)	Manual control over denoising steps (Fast)
Training Speed	Slow	Even slower (16x more pretraining [3])
Performance	Great	Similar

Masked Diffusion

Masked diffusion

- ▶ We follow the formalization of Shi et al..
- ▶ Forward process turns sequences of real tokens into mask tokens.
- ▶ Generation of new sequences requires a reverse process.

Masked diffusion

- ▶ We follow the formalization of Shi et al..
- ▶ Forward process turns sequences of real tokens into mask tokens.
- ▶ Generation of new sequences requires a reverse process.

Masked diffusion

- ▶ We follow the formalization of Shi et al..
- ▶ Forward process turns sequences of real tokens into mask tokens.
- ▶ Generation of new sequences requires a reverse process.

Forward process

- ▶ An uncorrupted token sequence is converted into a sequence of mask tokens [MASK] in T steps.
- ▶ The amount of tokens converted to mask tokens is determined by a masking schedule α_t .
 - ▶ We use a linear schedule, where approximately equal number of tokens are masked on every step.
 - ▶ Many other masking schedules exist, but all must begin from an uncorrupted token sequence and end at a masked sequence.

Forward process

- ▶ An uncorrupted token sequence is converted into a sequence of mask tokens [MASK] in T steps.
- ▶ The amount of tokens converted to mask tokens is determined by a masking schedule α_t .
 - ▶ We use a linear schedule, where approximately equal number of tokens are masked on every step.
 - ▶ Many other masking schedules exist, but all must begin from an uncorrupted token sequence and end at a masked sequence.

Reverse process

- ▶ Masked token sequences are converted into real tokens.
- ▶ Using Bayes rule on the forward process, a distribution over tokens can be obtained.
- ▶ Using a denoising network $\mu_\theta(y_t, x)$, we have the following distribution:

$$p_\theta(y_s | y_t, x) = \text{Cat}(y_s; \bar{R}^{\mu_\theta(y_t, x)}(t, s)^\top y_t)$$

$$\bar{R}^{\mu_\theta(y_t, x)}(t, s) = I + \frac{\alpha_s - \alpha_t}{1 - \alpha_t} e_m (\mu_\theta(y_t, x) - e_m)^\top \text{ and } s < t.$$

- ▶ Intuitively, if y_t is a mask token, we sample a new token from $\mu_\theta(y_t, x)$ with probability $p_{\text{unmask}}^{(s)} = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$, otherwise we keep the mask token. Unmasked tokens remain the same.

Reverse process

- ▶ Masked token sequences are converted into real tokens.
- ▶ Using Bayes rule on the forward process, a distribution over tokens can be obtained.
- ▶ Using a denoising network $\mu_\theta(y_t, x)$, we have the following distribution:

$$p_\theta(y_s | y_t, x) = \text{Cat}(y_s; \bar{R}^{\mu_\theta(y_t, x)}(t, s)^\top y_t)$$

$$\bar{R}^{\mu_\theta(y_t, x)}(t, s) = I + \frac{\alpha_s - \alpha_t}{1 - \alpha_t} e_m(\mu_\theta(y_t, x) - e_m)^\top \text{ and } s < t.$$

- ▶ Intuitively, if y_t is a mask token, we sample a new token from $\mu_\theta(y_t, x)$ with probability $p_{\text{unmask}}^{(s)} = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$, otherwise we keep the mask token. Unmasked tokens remain the same.

Reverse process

- ▶ Masked token sequences are converted into real tokens.
- ▶ Using Bayes rule on the forward process, a distribution over tokens can be obtained.
- ▶ Using a denoising network $\mu_\theta(y_t, x)$, we have the following distribution:

$$p_\theta(y_s | y_t, x) = \text{Cat}(y_s; \bar{R}^{\mu_\theta(y_t, x)}(t, s)^\top y_t)$$

$$\bar{R}^{\mu_\theta(y_t, x)}(t, s) = I + \frac{\alpha_s - \alpha_t}{1 - \alpha_t} e_m (\mu_\theta(y_t, x) - e_m)^\top \text{ and } s < t.$$

- ▶ Intuitively, if y_t is a mask token, we sample a new token from $\mu_\theta(y_t, x)$ with probability $p_{\text{unmask}}^{(s)} = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$, otherwise we keep the mask token. Unmasked tokens remain the same.

Reverse process

- ▶ Masked token sequences are converted into real tokens.
- ▶ Using Bayes rule on the forward process, a distribution over tokens can be obtained.
- ▶ Using a denoising network $\mu_\theta(y_t, x)$, we have the following distribution:

$$p_\theta(y_s|y_t, x) = \text{Cat}(y_s; \bar{R}^{\mu_\theta(y_t, x)}(t, s)^\top y_t)$$

$$\bar{R}^{\mu_\theta(y_t, x)}(t, s) = I + \frac{\alpha_s - \alpha_t}{1 - \alpha_t} e_m(\mu_\theta(y_t, x) - e_m)^\top \text{ and } s < t.$$

- ▶ Intuitively, if y_t is a mask token, we sample a new token from $\mu_\theta(y_t, x)$ with probability $p_{\text{unmask}}^{(s)} = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$, otherwise we keep the mask token. Unmasked tokens remain the same.

Evidence lower bound

- ▶ Usually maximum likelihood method is used to fit machine learning models, however, with diffusion models, the likelihood function is very inefficient to compute.
- ▶ Instead, we maximize the evidence lower bound (ELBO).
- ▶ For a masked diffusion model, the ELBO is computed as follows:

$$\begin{aligned}\log p_{\theta}(y|x) &\geq \mathcal{L}_{\theta}(y|x) \\ &= - \int_0^1 \frac{\alpha'_t}{1 - \alpha_t} \mathbb{E}_{q(y_t|y)} \left[\sum_{i=1}^L \mathbf{1}[y_t^{(i)} = [\text{MASK}]] (y^{(i)})^{\top} \log \mu_{\theta}(y_t, x)^{(i)} \right] dt\end{aligned}$$

- ▶ The integral and the expectation are computed with Monte-Carlo integration.

Evidence lower bound

- ▶ Usually maximum likelihood method is used to fit machine learning models, however, with diffusion models, the likelihood function is very inefficient to compute.
- ▶ Instead, we maximize the evidence lower bound (ELBO).
- ▶ For a masked diffusion model, the ELBO is computed as follows:

$$\begin{aligned}\log p_{\theta}(y|x) &\geq \mathcal{L}_{\theta}(y|x) \\ &= - \int_0^1 \frac{\alpha'_t}{1 - \alpha_t} \mathbb{E}_{q(y_t|y)} \left[\sum_{i=1}^L \mathbf{1}[y_t^{(i)} = [\text{MASK}]] (y^{(i)})^{\top} \log \mu_{\theta}(y_t, x)^{(i)} \right] dt\end{aligned}$$

- ▶ The integral and the expectation are computed with Monte-Carlo integration.

Evidence lower bound

- ▶ Usually maximum likelihood method is used to fit machine learning models, however, with diffusion models, the likelihood function is very inefficient to compute.
- ▶ Instead, we maximize the evidence lower bound (ELBO).
- ▶ For a masked diffusion model, the ELBO is computed as follows:

$$\begin{aligned}\log p_{\theta}(y|x) &\geq \mathcal{L}_{\theta}(y|x) \\ &= - \int_0^1 \frac{\alpha'_t}{1 - \alpha_t} \mathbb{E}_{q(y_t|y)} \left[\sum_{i=1}^L \mathbf{1}[y_t^{(i)} = [\text{MASK}]] (y^{(i)})^{\top} \log \mu_{\theta}(y_t, x)^{(i)} \right] dt\end{aligned}$$

- ▶ The integral and the expectation are computed with Monte-Carlo integration.

Evidence lower bound

- ▶ Usually maximum likelihood method is used to fit machine learning models, however, with diffusion models, the likelihood function is very inefficient to compute.
- ▶ Instead, we maximize the evidence lower bound (ELBO).
- ▶ For a masked diffusion model, the ELBO is computed as follows:

$$\begin{aligned}\log p_{\theta}(y|x) &\geq \mathcal{L}_{\theta}(y|x) \\ &= - \int_0^1 \frac{\alpha'_t}{1 - \alpha_t} \mathbb{E}_{q(y_t|y)} \left[\sum_{i=1}^L \mathbf{1}[y_t^{(i)} = [\text{MASK}]] (y^{(i)})^{\top} \log \mu_{\theta}(y_t, x)^{(i)} \right] dt\end{aligned}$$

- ▶ The integral and the expectation are computed with Monte-Carlo integration.

Sampling

1. Initialize a token sequence to mask tokens.
2. For T steps, repeat the following:
 - 2.1 For each masked token, compute

$$\log p_i \leftarrow \log \left(\frac{\alpha_s - \alpha_t}{1 - \alpha_t} \mu_\theta(y_t, x)^{(i)} + \frac{1 - \alpha_s}{1 - \alpha_t} e_m \right).$$

- 2.2 Replace the masked token with the token sampled from $\text{Cat}(y_{t-1}; p_i)$.

Sampling

1. Initialize a token sequence to mask tokens.
2. For T steps, repeat the following:
 - 2.1 For each masked token, compute

$$\log p_i \leftarrow \log \left(\frac{\alpha_s - \alpha_t}{1 - \alpha_t} \mu_\theta(y_t, x)^{(i)} + \frac{1 - \alpha_s}{1 - \alpha_t} e_m \right).$$

- 2.2 Replace the masked token with the token sampled from $\text{Cat}(y_{t-1}; p_i)$.

Reinforcement learning

Reinforcement learning

- ▶ Reinforcement learning for an agent to improve in a task by exploring an environment and getting feedback for its actions.
- ▶ The environment is often given by a Markov Decision Process (MDP), which has the following components:
 - ▶ States, actions, policy
 - ▶ Transition function, initial state distribution, reward function
- ▶ In the environment, the agent takes actions according to the policy and attempts to maximize the cumulative reward over a trajectory.

Reinforcement learning

- ▶ Reinforcement learning for an agent to improve in a task by exploring an environment and getting feedback for its actions.
- ▶ The environment is often given by a Markov Decision Process (MDP), which has the following components:
 - ▶ States, actions, policy
 - ▶ Transition function, initial state distribution, reward function
- ▶ In the environment, the agent takes actions according to the policy and attempts to maximize the cumulative reward over a trajectory.

Reinforcement learning

- ▶ Reinforcement learning for an agent to improve in a task by exploring an environment and getting feedback for its actions.
- ▶ The environment is often given by a Markov Decision Process (MDP), which has the following components:
 - ▶ States, actions, policy
 - ▶ Transition function, initial state distribution, reward function
- ▶ In the environment, the agent takes actions according to the policy and attempts to maximize the cumulative reward over a trajectory.

Proximal policy optimization

- ▶ Policy gradient theorem can be used to optimize the parameters of a policy [5].
- ▶ One algorithm that is often used is the so-called proximal policy optimization (PPO):

$$L(\theta) = \mathbb{E}_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)],$$

where $\hat{A}_t = R - V(S_t)$ is the advantage function and $\rho_t = \frac{\pi_\theta(A_t|S_t)}{\pi_{\theta_{\text{old}}}(A_t|S_t)}$ is the importance sampling ratio used in PPO.

Proximal policy optimization

- ▶ Policy gradient theorem can be used to optimize the parameters of a policy [5].
- ▶ One algorithm that is often used is the so-called proximal policy optimization (PPO):

$$L(\theta) = \mathbb{E}_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)],$$

where $\hat{A}_t = R - V(S_t)$ is the advantage function and $\rho_t = \frac{\pi_\theta(A_t|S_t)}{\pi_{\theta_{\text{old}}}(A_t|S_t)}$ is the importance sampling ratio used in PPO.

Denoising diffusion policy optimization

- ▶ Denoising diffusion policy optimization (DDPO) is an algorithm based on PPO designed for continuous diffusion models.
- ▶ Can be easily adapted to masked diffusion models.
- ▶ The objective is then as follows:

$$\mathcal{J}(\theta) = L(\theta) - \beta \mathbb{E}_i [\text{KL}(\pi_\theta \parallel \pi_{\text{ref}} \mid s_i)] + \gamma \mathbb{E}_i [H(\pi_\theta \mid s_i)],$$

where the KL divergence and the entropy are computed with the new and reference policies.

Denoising diffusion policy optimization

- ▶ Denoising diffusion policy optimization (DDPO) is an algorithm based on PPO designed for continuous diffusion models.
- ▶ Can be easily adapted to masked diffusion models.
- ▶ The objective is then as follows:

$$\mathcal{J}(\theta) = L(\theta) - \beta \mathbb{E}_i [\text{KL}(\pi_\theta \parallel \pi_{\text{ref}} \mid s_i)] + \gamma \mathbb{E}_i [H(\pi_\theta \mid s_i)],$$

where the KL divergence and the entropy are computed with the new and reference policies.

Denoising diffusion policy optimization

- ▶ Denoising diffusion policy optimization (DDPO) is an algorithm based on PPO designed for continuous diffusion models.
- ▶ Can be easily adapted to masked diffusion models.
- ▶ The objective is then as follows:

$$\mathcal{J}(\theta) = L(\theta) - \beta \mathbb{E}_i [\text{KL}(\pi_\theta \parallel \pi_{\text{ref}} \mid s_i)] + \gamma \mathbb{E}_i [H(\pi_\theta \mid s_i)],$$

where the KL divergence and the entropy are computed with the new and reference policies.

Methodology

Tokenization

- ▶ Deep learning models cannot parse natural language, so it must be transformed into a machine-readable form.
- ▶ Chess positions can be represented with the Forsyth-Edwards Notation (FEN):

`rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR` `b` `KQkq` `e3` `0` `1`
board side castling en passant halfmove fullmove

- ▶ We tokenize the positions using FENs.

10, 8, 9, 11, 12, 9, 8, 10, 7, 7, 7, 7, 7, 7, 7, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, , 14, 16, 17, 18, 19, 33, 23, 37, 38, 37, 37, 39
0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 4, 2, 3, 5, 6, 3, 2, 4
board side castling en passant halfmove fullmove

- ▶ Additionally, we concatenate the tokenized version of the best move to the FEN tokens.

Tokenization

- ▶ Deep learning models cannot parse natural language, so it must be transformed into a machine-readable form.
- ▶ Chess positions can be represented with the Forsyth-Edwards Notation (FEN):

rnbgkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1
board side castling en passant halfmove fullmove

- ▶ We tokenize the positions using FENs.

10, 8, 9, 11, 12, 9, 8, 10, 7, 7, 7, 7, 7, 7, 7, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 4, 2, 3, 5, 6, 3, 2, 4 , 14 , 16, 17, 18, 19, 33, 23 , 37, 38 , 37, 37, 39
board side castling en passant halfmove fullmove

- ▶ Additionally, we concatenate the tokenized version of the best move to the FEN tokens.

Tokenization

- ▶ Deep learning models cannot parse natural language, so it must be transformed into a machine-readable form.
- ▶ Chess positions can be represented with the Forsyth-Edwards Notation (FEN):

$\underbrace{\text{rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR}}_{\text{board}}$
 $\underbrace{\text{b}}_{\text{side}}$
 $\underbrace{\text{KQkq}}_{\text{castling}}$
 $\underbrace{\text{e3}}_{\text{en passant}}$
 $\underbrace{0}_{\text{halfmove}}$
 $\underbrace{1}_{\text{fullmove}}$

- ▶ We tokenize the positions using FENs.

$$\underbrace{10, 8, 9, 11, 12, 9, 8, 10, 7, 7, 7, 7, 7, 7, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 4, 2, 3, 5, 6, 3, 2, 4}_{\text{board}}, \underbrace{14}_{\text{side}}, \underbrace{16, 17, 18, 19}_{\text{castling}}, \underbrace{33, 23}_{\text{en passant}}, \underbrace{37, 38}_{\text{halfmove}}, \underbrace{37, 37, 39}_{\text{fullmove}}$$

Tokenization

- ▶ Deep learning models cannot parse natural language, so it must be transformed into a machine-readable form.
- ▶ Chess positions can be represented with the Forsyth-Edwards Notation (FEN):

rnbgkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1
board side castling en passant halfmove fullmove

- ▶ We tokenize the positions using FENs.

10, 8, 9, 11, 12, 9, 8, 10, 7, 7, 7, 7, 7, 7, 7, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, , 14 , 16, 17, 18, 19, 33, 23 , 37, 38 , 37, 37, 39
0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 4, 2, 3, 5, 6, 3, 2, 4
board side castling en passant halfmove fullmove

- ▶ Additionally, we concatenate the tokenized version of the best move to the FEN tokens.

Chess puzzles

- ▶ We define chess puzzles similarly to Feng et al..
- ▶ A position is considered a puzzle if it passes the following criteria:
 - ▶ Legality
 - ▶ No two kings for one player etc.
 - ▶ Uniqueness of solution
 - ▶ Solution is unique if the winning chances after the best move are substantially better than after the second-best move.
 - ▶ Counter-intuitiveness

Chess puzzles

- ▶ We define chess puzzles similarly to Feng et al..
- ▶ A position is considered a puzzle if it passes the following criteria:
 - ▶ Legality
 - ▶ No two kings for one player etc.
 - ▶ Uniqueness of solution
 - ▶ Solution is unique if the winning chances after the best move are substantially better than after the second-best move.
 - ▶ Counter-intuitiveness

Chess puzzles: Counter-intuitiveness

- ▶ The counter-intuitiveness of a puzzle is defined as the linear combination of the captured material value and the critical point of the search, which is the depth, where Stockfish finds the correct move.

$$\begin{cases} r_{\text{counter-intuitive}}(s) = 0.8 \cdot \frac{d_{\text{cp}} - 1}{50} - 0.1 \cdot \frac{v_{\text{capturedMaterial}}}{9}, \\ \mathbb{I}_{\text{counter-intuitive}}(s) = r_{\text{counter-intuitive}}(s) > \tau_{\text{cnt}}, \end{cases}$$

- ▶ Counter-intuitiveness and uniqueness are inversely correlated. Passing both criteria is hard, as Stockfish is so good.

Chess puzzles: Counter-intuitiveness

- ▶ The counter-intuitiveness of a puzzle is defined as the linear combination of the captured material value and the critical point of the search, which is the depth, where Stockfish finds the correct move.

$$\begin{cases} r_{\text{counter-intuitive}}(s) = 0.8 \cdot \frac{d_{\text{cp}} - 1}{50} - 0.1 \cdot \frac{v_{\text{capturedMaterial}}}{9}, \\ \mathbb{I}_{\text{counter-intuitive}}(s) = r_{\text{counter-intuitive}}(s) > \tau_{\text{cnt}}, \end{cases}$$

- ▶ Counter-intuitiveness and uniqueness are inversely correlated. Passing both criteria is hard, as Stockfish is so good.

Reward function

- ▶ Puzzle criterion
- ▶ Theme matching
- ▶ Diversity filtering
 - ▶ Levenshtein distance between positions in current and preceding batches over fen and principal variation [6].
- ▶ Entropy filtering
- ▶ Piece count regularization
 - ▶ Counts of either player's pieces should not exceed the counts at the beginning of the game to ensure realism.

Results

Supervised results

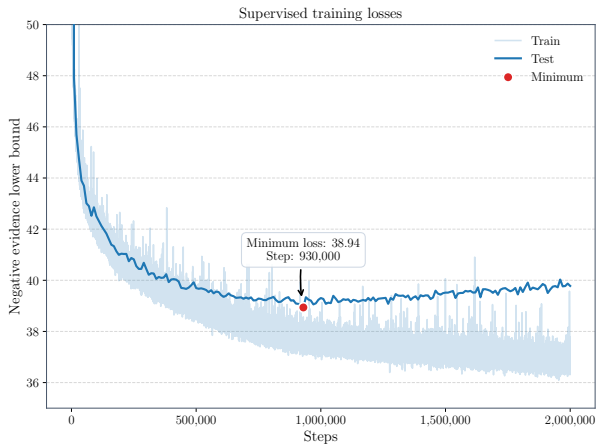


Figure: The negative evidence lower bound as a function of the training steps. After around 1,000,000, we observe overfitting.

Supervised results

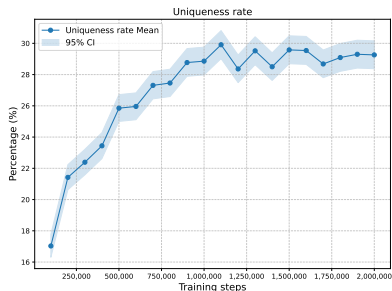
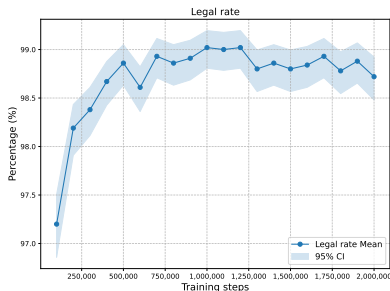


Figure: Evolution of the primary quality metrics with context from the test set during supervised pretraining. We omit the first data point, as it is identically zero for all metrics.

Supervised results

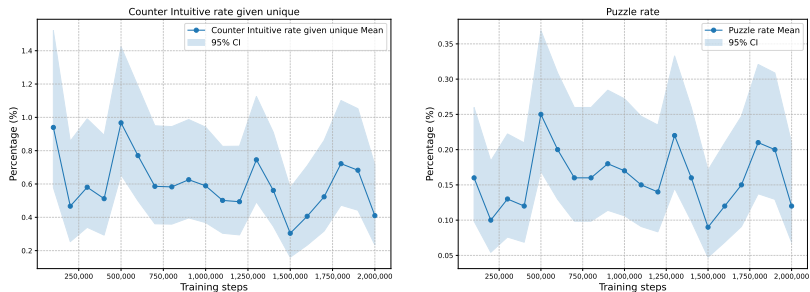


Figure: Evolution of the primary quality metrics with context from the test set during supervised pretraining. We omit the first data point, as it is identically zero for all metrics.

Supervised results

Table: The final metrics after supervised training with different context distributions. The sampling is performed with a temperature 1.0 and 256 denoising steps. The errors are computed as the 95% confidence intervals of the wanted proportion [1].

Sampling	Legal	Unique	Counter-intuitive unique	Puzzle	Theme
block	98.97 ± 0.044	27.08 ± 0.19	0.63 ± 0.018	0.17 ± 0.018	85.53 ± 0.18
normal	98.95 ± 0.045	28.75 ± 0.20	0.61 ± 0.018	0.18 ± 0.019	86.27 ± 0.19
Lichess puzzles	100.0 ± 0.0	83.58 ± 0.23	1.55 ± 0.070	1.29 ± 0.070	100.0 ± 0.0

Reinforcement learning results

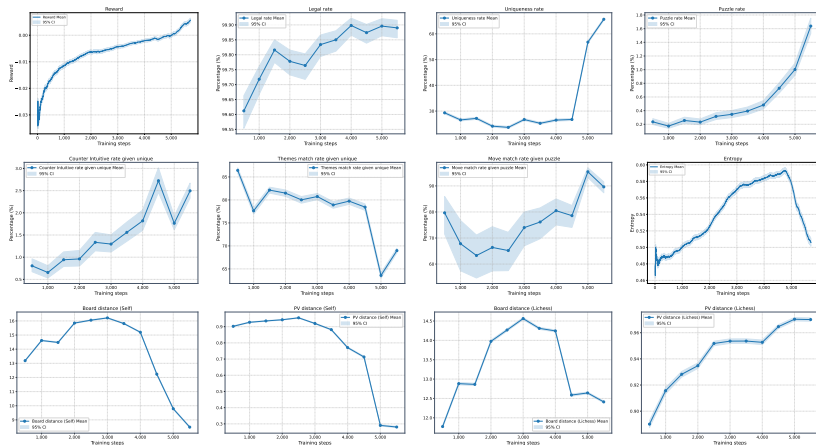


Figure: The training progress during reinforcement learning. We observe model collapse after around 4,500 steps.

Reinforcement learning results

Table: The final metrics after RL training for 3,500 steps. The sampling is performed with a temperature 1.0 and 256 denoising steps. The errors are computed as the 95% confidence intervals of the wanted proportion [1].

Sampling	Legal	Unique	Counter-intuitive unique	Puzzle	Theme
normal	99.87 ± 0.01	25.28 ± 0.09	1.45 ± 0.012	0.37 ± 0.012	78.72 ± 0.08

Conclusion

Conclusion

- ▶ Masked diffusion models are an effective alternative to LLMs.
- ▶ Puzzle generation can be accurately conditioned on specific tactical themes.
- ▶ Performance of Masked diffusion models can be meaningfully improved through reinforcement learning, doubling the puzzle rate from 0.18% to 0.37%.
- ▶ Integrating the auxiliary task of predicting the best move enhances the generative capabilities.
- ▶ Future directions include generalization to other domains, exploring alternative RL algorithms (e.g., ESPO) and including chain-of-thought to puzzle generation.

Conclusion

- ▶ Masked diffusion models are an effective alternative to LLMs.
- ▶ Puzzle generation can be accurately conditioned on specific tactical themes.
- ▶ Performance of Masked diffusion models can be meaningfully improved through reinforcement learning, doubling the puzzle rate from 0.18% to 0.37%.
- ▶ Integrating the auxiliary task of predicting the best move enhances the generative capabilities.
- ▶ Future directions include generalization to other domains, exploring alternative RL algorithms (e.g., ESPO) and including chain-of-thought to puzzle generation.

Conclusion

- ▶ Masked diffusion models are an effective alternative to LLMs.
- ▶ Puzzle generation can be accurately conditioned on specific tactical themes.
- ▶ Performance of Masked diffusion models can be meaningfully improved through reinforcement learning, doubling the puzzle rate from 0.18% to 0.37%.
- ▶ Integrating the auxiliary task of predicting the best move enhances the generative capabilities.
- ▶ Future directions include generalization to other domains, exploring alternative RL algorithms (e.g., ESPO) and including chain-of-thought to puzzle generation.

Conclusion

- ▶ Masked diffusion models are an effective alternative to LLMs.
- ▶ Puzzle generation can be accurately conditioned on specific tactical themes.
- ▶ Performance of Masked diffusion models can be meaningfully improved through reinforcement learning, doubling the puzzle rate from 0.18% to 0.37%.
- ▶ Integrating the auxiliary task of predicting the best move enhances the generative capabilities.
- ▶ Future directions include generalization to other domains, exploring alternative RL algorithms (e.g., ESPO) and including chain-of-thought to puzzle generation.

Conclusion

- ▶ Masked diffusion models are an effective alternative to LLMs.
- ▶ Puzzle generation can be accurately conditioned on specific tactical themes.
- ▶ Performance of Masked diffusion models can be meaningfully improved through reinforcement learning, doubling the puzzle rate from 0.18% to 0.37%.
- ▶ Integrating the auxiliary task of predicting the best move enhances the generative capabilities.
- ▶ Future directions include generalization to other domains, exploring alternative RL algorithms (e.g., ESPO) and including chain-of-thought to puzzle generation.

References

- [1] Lawrence D Brown, T Tony Cai, and Anirban DasGupta. Interval estimation for a binomial proportion. *Statistical Science*, 16(2):101–133, 2001.
- [2] Xidong Feng, Vivek Veeriah, Marcus Chiam, Michael D Dennis, Federico Barbero, Johan Obando-Ceron, Jiaxin Shi, Satinder Singh, Shaobo Hou, Nenad Tomasev, and Tom Zahavy. Generating creative chess puzzles. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=TNZse5q2Tr>.

References (cont.)

- [3] Shen Nie, Fengqi Zhu, Chao Du, Tianyu Pang, Qian Liu, Guangtao Zeng, Min Lin, and Chongxuan Li. Scaling up masked diffusion models on text. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=WNvwwK0tut>.
- [4] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 103131–103167. Curran Associates, Inc., 2024. doi: 10.52202/079017-3277. URL https://proceedings.neurips.cc/paper_files/paper/

References (cont.)

2024/file/
bad233b9849f019ae5e5cc60cef70f-Paper-Conference.
pdf.

- [5] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.
- [6] Li Yujian and Liu Bo. A normalized levenshtein distance metric. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(6):1091–1095, 2007. doi: 10.1109/TPAMI.2007.1078.

Example puzzles

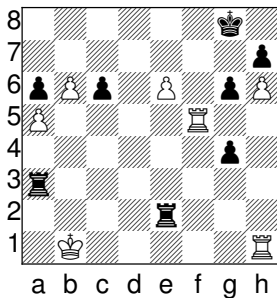


Figure: Example 1

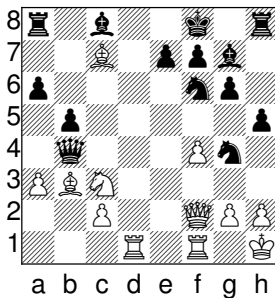


Figure: Example 2

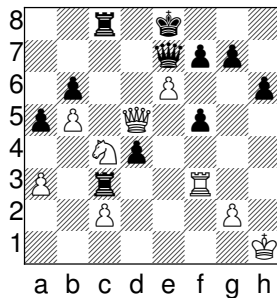


Figure: Example 3

Example puzzles

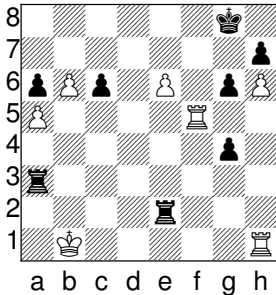


Figure: Example 1¹

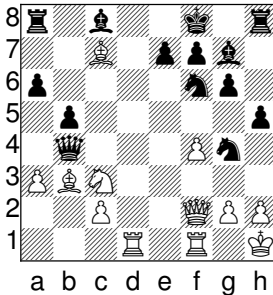


Figure: Example 2²

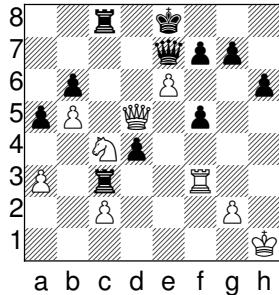


Figure: Example 3³

¹Principal variation: f5b5 c6b5 b6b7

²Principal variation: f2d2 b4d6 c7d6

³Principal variation: e6f7 e8f8 c4e5 e7e5 d5e5